0115. Distinct Subsequences

Hard · 2-D Dynamic Programming

leetcode.com/problems/distinct-subsequences

Problem

- Count the distinct subsequences of s that equal t.

Intuition

- Dp[i][j] = ways for s[0..i) to form t[0..j).
- We can always skip s[i-1]: dp[i-1][j]. If s[i-1] == t[j-1], we can also use it: + dp[i-1][j-1].

Brute Force

- Enumerate subsequences of s: O(2^n).

Optimal Approach

- Dp[i][0] = 1 for all i (empty t is formed one way).
- Fill with the rule above; compress to 1-D iterating j downward.

Worked Example

```
s = "rabbbit", t = "rabbit".

the three b's in s; t needs two -> choose 2 of 3 = 3 ways
answer 3
```

Java Solution

```java
public class DistinctSubsequences {
    public int numDistinct(String s, String t) {
        int n = t.length();
        long[] dp = new long[n + 1];
        dp[0] = 1;
        for (int i = 0; i < s.length(); i++)
            for (int j = n; j >= 1; j--)
                if (s.charAt(i) == t.charAt(j - 1)) dp[j] += dp[j - 1];
        return (int) dp[n];
    }
}
```

Complexity

- Time: O(m n). Space: O(n).

Pitfalls

- Iterate j downwards in the 1-D version so dp[j-1] is still the previous row's value.
- Intermediate counts can be large; the problem guarantees the answer fits in 32 bits, but use long internally to be safe.